

Race master 4D: racist game

Practical work documentation

Michael Gustavo & Danila Malyshev

September 67, 2026

1 Clarified task

The project is a C++ program for simulating and displaying traffic at a four-way intersection. It models cars moving on several lanes in both directions, traffic lights for straight, left, and right movements, and pedestrians crossing the roads. The purpose is to observe queues and how they change when traffic and signal settings are changed.

The program has two signal modes. In static mode, the green, yellow, and all-red pedestrian phase use selected durations. In automatic mode, green and pedestrian phase times depend on the waiting queues. The user can also change signal visibility, car speed limits, and the randomized car-arrival interval. Car speed limits and arrival intervals are sampled from uniform distributions within the selected ranges. In automatic mode, pedestrian arrivals also use a uniformly sampled interval, from 2.5 to 7 seconds.

The current implementation builds four approaches, each with six lanes. Three lanes carry incoming traffic and three carry outgoing traffic. New cars are placed in a randomly selected available incoming lane, but each lane has a fixed movement, so cars do not choose a random turn. The model includes car following and signal stopping, but it does not yet include lane changes. Collisions are not simulated.

For each new car, the speed limit is sampled uniformly from the selected speed range. Its initial speed is sampled from the selected minimum up to that vehicle's speed limit. Cars follow route waypoints, slow down behind other vehicles, stop for signals, and wait when a left turn conflicts with opposing traffic or the exit lane is blocked. A car is removed after completing its route.

2 Class diagram

The diagram shows the main active model classes and their relationships. Arrows marked with a number show how many objects are normally created. The dashed line is a signal reference, and the open triangle marks inheritance.

The **Crossroad** class exists in the source but is currently a placeholder, so it is not connected to the active model in the diagram.

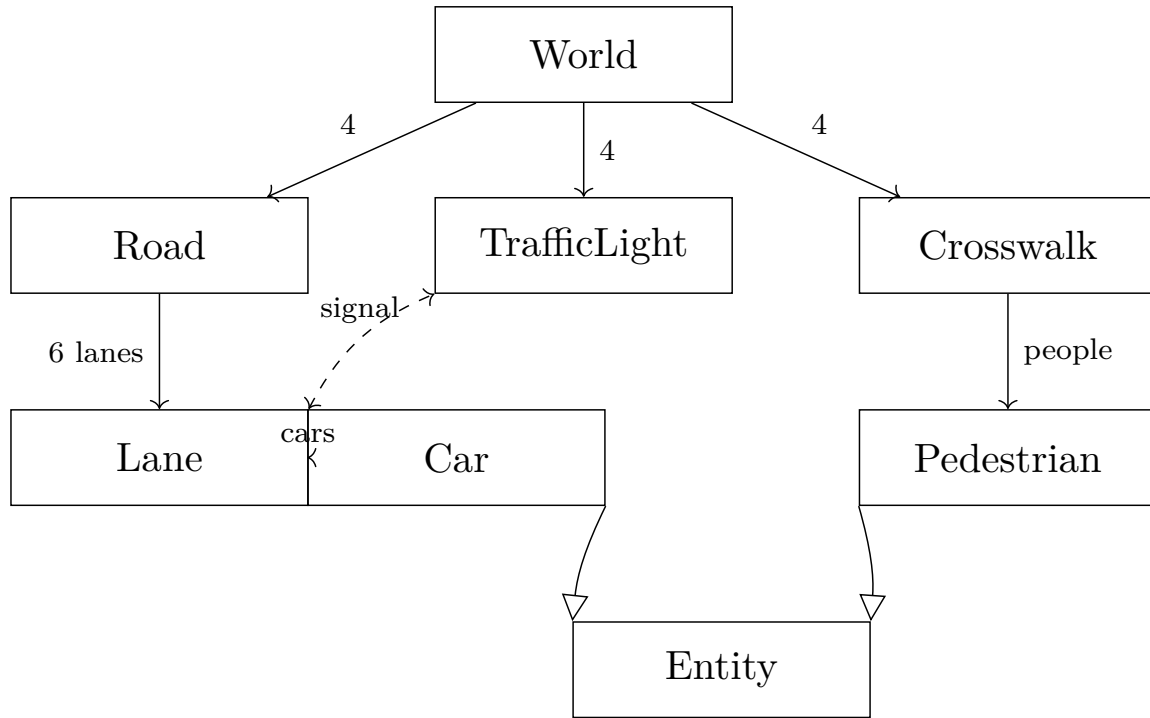


Figure 1: Main classes used by the simulation.

3 Main class specifications

The following table gives a short description of the main classes and their public operations. The implementations are mostly in **objects.h**.

Class	Purpose	Main operations
Entity	Common position, direction, velocity, speed, acceleration, and speed limit for moving objects.	GetPos() , GetDirection() , GetSpeedLimit() , and setters for speed and acceleration.
Car	Inherits from Entity . Stores its lane, route, turn, current waypoint, and whether it is still active. Follows the route and reacts to signals and other cars.	Car(Lane*) , Update(dt) , IsAlive() , Turn() , StopAt() .
Pedestrian	Inherits from Entity . Stores a crossing path and moves along it at a fixed model speed of 5 km/h.	SetPath(from,to) , Update(dt) , IsAlive() .
Lane	Stores road geometry, route waypoints, cars, a movement type, and references to a signal and exit lane. Also checks whether there is room for another car.	Build(...) , HasRoom() , Update(dt) , CurrentSignal() .
Road	Builds and owns the six lanes for one approach.	Build(outward,n) , Update(dt) , GetLanes() .
TrafficLight	Stores the signal position, orientation, and current signal state, including protected left-turn states.	GetSignal() , SetSignal() , GetPos() .
Crosswalk	Stores crossing endpoints and its pedestrians. It can create a pedestrian, check whether the crossing is busy, and update the people on it.	Build(a,b) , Spawn() , Busy() , Update(dt) .
World	Builds and updates the four roads, lights, and crosswalks. It also controls signal phases, vehicle arrivals, and automatic pedestrian demand.	Build() , Update(dt) , SetAutomaticSignals(bool) , WaitingVehiclesForActivePhase() .
Crossroad	Placeholder for intersection geometry. It currently has no active behavior.	No active public interface.
background_manager	Loads the available background textures and changes the image shown behind the simulation.	Init_textures() , Get_background(...) , Next_background(...) .

4 Tools and libraries

Item	Used in the project
Programming language	C++17.
Graphics and window	SFML 3.0 (Graphics, Window, and System).
User interface	ImGui-SFML.
Build system	CMake 3.24 or later; the CMake project also checks for OpenGL.
Development environment	Project source is in the Replit workspace. No separate C++ IDE is specified in the project files.
Report editor	The report is set up for LuaLaTeX and can be opened in TeXstudio.

5 File structure

The source files are arranged as follows:

```
crossroads/  
|-- main.cpp  
|-- app.h  
|-- app.cpp  
|-- objects.h  
|-- objects.cpp  
|-- CMakeLists.txt  
`-- project_documentation.tex
```

main.cpp starts the application by calling **app::run()**. **app.h** contains drawing declarations and the background manager. **app.cpp** creates the window, processes input, updates the world, and draws the simulation and ImGui controls. **objects.h** contains the model classes and most of their inline implementations. **objects.cpp** includes that header for the CMake target. **CMakeLists.txt** lists the program sources and external libraries.

To build the program on a system with the required libraries installed, run:

```
cmake -S crossroads -B crossroads/build  
cmake --build crossroads/build
```

6 User interface

The program opens a 1200 by 800 pixel window. It displays the roads, cars, traffic lights, crosswalks, and pedestrians. The ImGui panel shows the active signal mode, the number of vehicles waiting for the current phase, and the pedestrian queue in automatic mode. A separate arrow section on the traffic light indicates a protected left-turn signal.

The settings in the panel are listed below. Static signal durations only affect static mode. Changes to the speed range affect new cars.

Control	Default and range
Signal mode	Static preset or automatic.
Green duration	12 s by default; slider range 1–90 s.
Yellow duration	3 s by default; slider range 1–15 s.
Red / walk duration	12 s by default; slider range 1–90 s. This is the all-red pedestrian phase.
Signal visibility	30 m by default; slider range 5–100 m.
Car speed range	30–120 km/h by default; values can be set from 5 to 160 km/h.
Car arrival interval	4–8 s by default. Minimum slider: 1–30 s; maximum: from the selected minimum up to 60 s.

The world attempts to place one car on each road at startup. It then draws one random interval for the whole intersection and attempts to add one car to each road after that interval. Thus, all four roads use the same period. The program chooses an incoming lane with room; it skips an arrival if all lanes on that road are full. The **Add car** button attempts to add one car to every incoming lane with room. **Next background** changes the background image. The background manager expects image files under `../assets/`.

7 GitHub repository

The GitHub repository can be found [here](#). Main branch was hardly used (main is for losers), all the significant commits are located at the "Misha" branch.